

Fluxo GitFlow

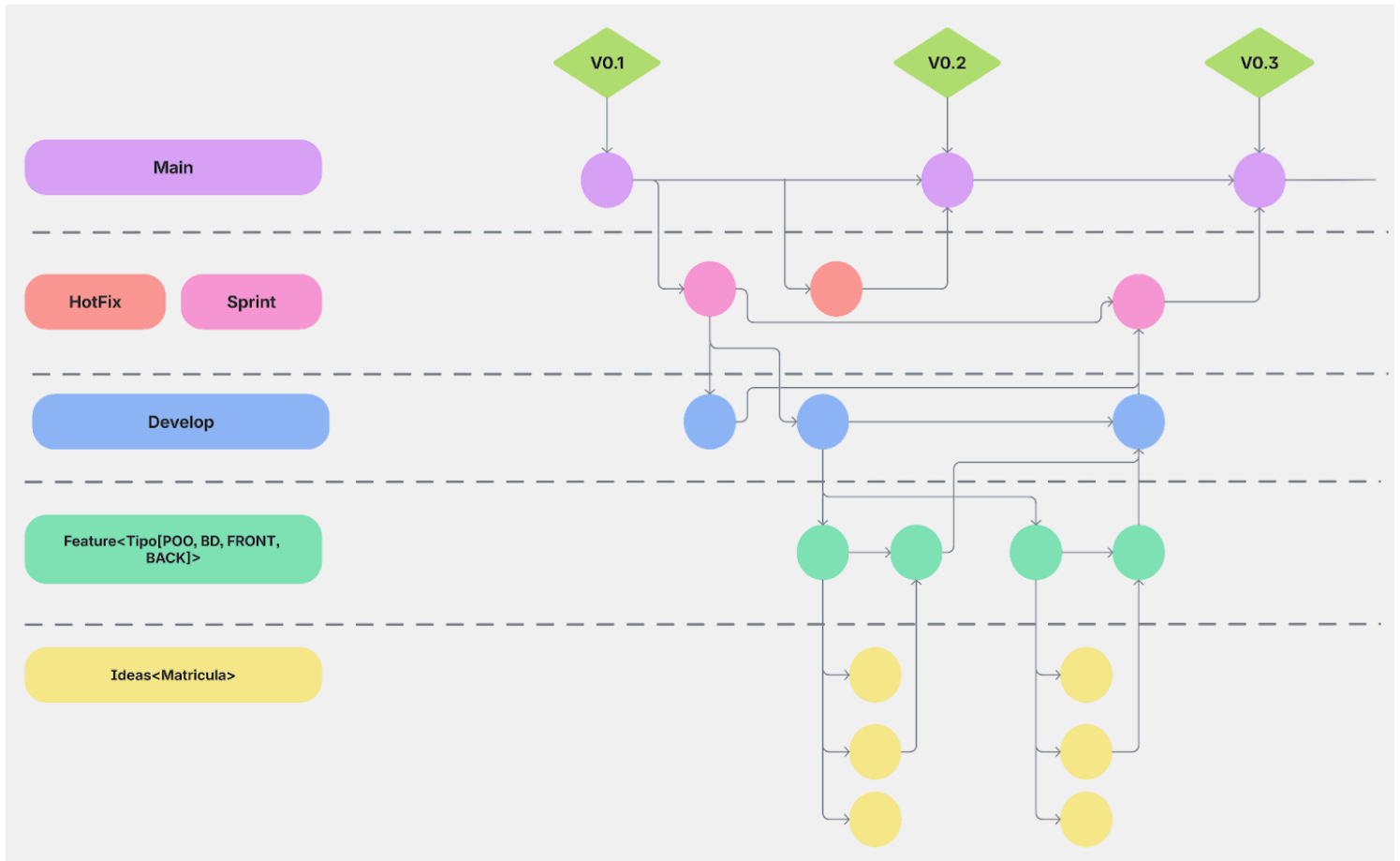
1. Objetivo

Este documento descreve o funcionamento do fluxo de branches Git adotado pelo projeto, detalhando responsabilidades, momentos de criação, permissões de merge e padrões de nomenclatura. O objetivo é garantir controle de versão, rastreabilidade e qualidade do software, evitando alterações indevidas e mantendo um processo claro para todos os envolvidos.

2. Regras Gerais

- É estritamente proibido alterar código diretamente na branch **main**.
- Todo merge para **main** ou **develop** deve ser realizado por **desenvolvedores de cargo sênior** ou superiores, com **aprovação obrigatória de Analistas de Qualidade** via Pull Request (PR).
- Todos os casos de uso, códigos de erro e registros de branches devem ser documentados e expostos no **ClickUp**, garantindo rastreabilidade.
- Branches descartáveis (**ideas**) serão removidas ao final de cada sprint, mas salvas como **tags no GitHub** para fins de métricas. -> TALVEZ ALTERAR

3. Estrutura de Branches



3.1 Main

- **Função:** representa a versão estável do software em desenvolvimento.
- **Origem de ramificações:**
 - **Sprint<nºSprint>** para releases planejadas.
 - **Hotfix<numeroDoErro>** para correções urgentes.
- **Responsáveis:** apenas **desenvolvedores sêniores** podem realizar merge, com aval de **Analistas de Qualidade** via PR.
- **Padrão léxico:** **main**.

3.2 Sprint

- **Função:** integrar casos de uso desenvolvidos em um mesmo ciclo de sprint. Serve também para gerar releases.
- **Criação:** realizada a partir de **main**.
- **Responsáveis:** desenvolvedores sêniores e designers.
- **Destino do merge:**
 - De volta para **main** (gerando nova versão estável).
 - De volta para **develop** (mantendo sincronia com desenvolvimento).
- **Padrão léxico:** **Sprint<n°Sprint>**
 - Exemplo: **Sprint01**.

3.3 Develop

- **Função:** branch de integração contínua das features que implementam casos de uso documentados.
- **Criação:** feita a partir de **main**.
- **Responsáveis:** mantida pelos **desenvolvedores sêniores e designers**.
- **Destino do merge:**
 - Recebe merges de **feature** (desenvolvimento de casos de uso).
 - Recebe merges de **sprint** e **hotfix** após liberação.
- **Padrão léxico:** **Develop<CodigoCasoDeUso>**
 - Exemplo: **DevelopCSU02**.

3.4 Hotfix

- **Função:** usada em situações de correções urgentes em produção, sem necessidade de criar uma branch **develop**.
- **Criação:** feita a partir de **main**.
- **Responsáveis:** apenas **desenvolvedores sêniores** podem criar e manter.
- **Destino do merge:**
 - De volta para **main** (corrigindo a versão).
 - De volta para **develop** (mantendo consistência).
- **Condições:** merge só após aprovação de **Analistas de Qualidade** via PR.

- **Padrão léxico:** **Hotfix<numeroDoErro>**
 - Exemplo: **Hotfix03.**

3.5 Feature

- **Função:** desenvolver funcionalidades específicas (casos de uso).
- **Criação:** feita a partir de **develop**.
- **Responsáveis:**
 - **Devs sêniores:** criam e mantêm features dos tipos **BACK, POO, BD**.
 - **Designers de jogo:** criam features do tipo **FRONT**.
 - **Devs júnior:** podem criar **ideas** a partir das features para propor soluções.
- **Destino do merge:** volta para **develop** após conclusão e validação.
- **Padrão léxico:** **Feature<Tipo[POO, BD, FRONT, BACK], CódigoCasoDeUso>**
 - Exemplo: **FeatureFrontCSU04.**

3.6 Ideas

- **Função:** ambiente de experimentação, onde **Devs júnior** propõem soluções.
- **Criação:** feita a partir de uma **feature**.
- **Destino do merge:** retorna para a **feature**, após avaliação da melhor solução.
- **Validação:** apenas a solução aprovada segue para **develop**.
- **Destino final:** branches **ideas** são apagadas ao final da sprint, mas salvas como tags para métricas. -> TALVEZ ALTERAR
- **Padrão léxico:** **Ideas<Matricula, Tipo, CódigoCasoDeUso>**
 - Exemplo: **Ideas5078CSU05.**

4. Fluxo Resumido

- O trabalho parte de **main** (estável).
- Criam-se **sprint** (para releases) ou **hotfix** (para correções urgentes).

- O desenvolvimento contínuo ocorre em **develop**.
- A partir de develop, nascem as **features** específicas por tipo de caso de uso.
- Dentro de uma feature, **juniors criam ideas** para propor soluções.
- As melhores soluções sobem da idea → feature → develop.
- Ao final da sprint, develop é integrado em sprint, que depois vai para main (nova versão).
- Todo merge em branches críticas (main, develop) exige PR com aprovação de Analistas de Qualidade.